

국민대학교 AI빅데이터 분석 경진대회

김민욱, 김지현, 민단아, 한유승

CONTENTS

1 대회 개요

2 데이터 소개

3 정확도 추이

4 코드 설명

5 모델 개요

6 version 변화

DACON

커뮤니티

대회

학습

랭킹

더보기

제3회 국민대학교 AI빅데이터 분석 경진대회

알고리즘 | 정형 | 시계열 | 무역 | 분류 | 회귀

₩ 상금 700만 원

🕒 2025.11.10 ~ 2025.11.28 09:59 + Google Calendar

👤 1,426명 📅 종료까지 D-8

대회안내

데이터

코드 공유

토크

리더보드

팀

제출

개요

평가

규칙

일정

상금

[배경]

국민대학교 경영대학원 AI빅데이터/디지털마케팅전공과 경영대학에서 '제3회 국민대학교 AI빅데이터 분석 경진대회'를 개최합니다.

이번 대회는 국민대학교 경영대학원과 한국기계산업진흥회(KOAMI)이 공동으로 주최하여 품목 간 무역 연동성 기술의 응용을 주제로 진행됩니다.

참가자는 100개 수입 품목의 월별 무역 데이터를 분석하여 공행성이 있는 품목 쌍을 판별하고, 선행 품목의 흐름을 바탕으로 향후 달 무역량을 예측하는 알고리즘을 개발하게 됩니다.

이를 통해 무역 데이터에서의 품목 간 구조적 관계를 탐색하고, 실무에 적용 가능한 예측 기반 의사결정 지원 도구를 개발할 수 있을 것으로 기대합니다.

대회 소개

- 국민대 경영대학·AI대학원 주관 데이터 분석 경진대회
- 실제 무역 데이터를 기반으로 한 실무형 분석 문제

대회 목적

- 무역 데이터 구조 및 HS코드 기반 품목 분석 경험 제공
- AI·머신러닝 기반 예측/분석 역량 강화
- 실제 산업 문제 해결 경험 제공

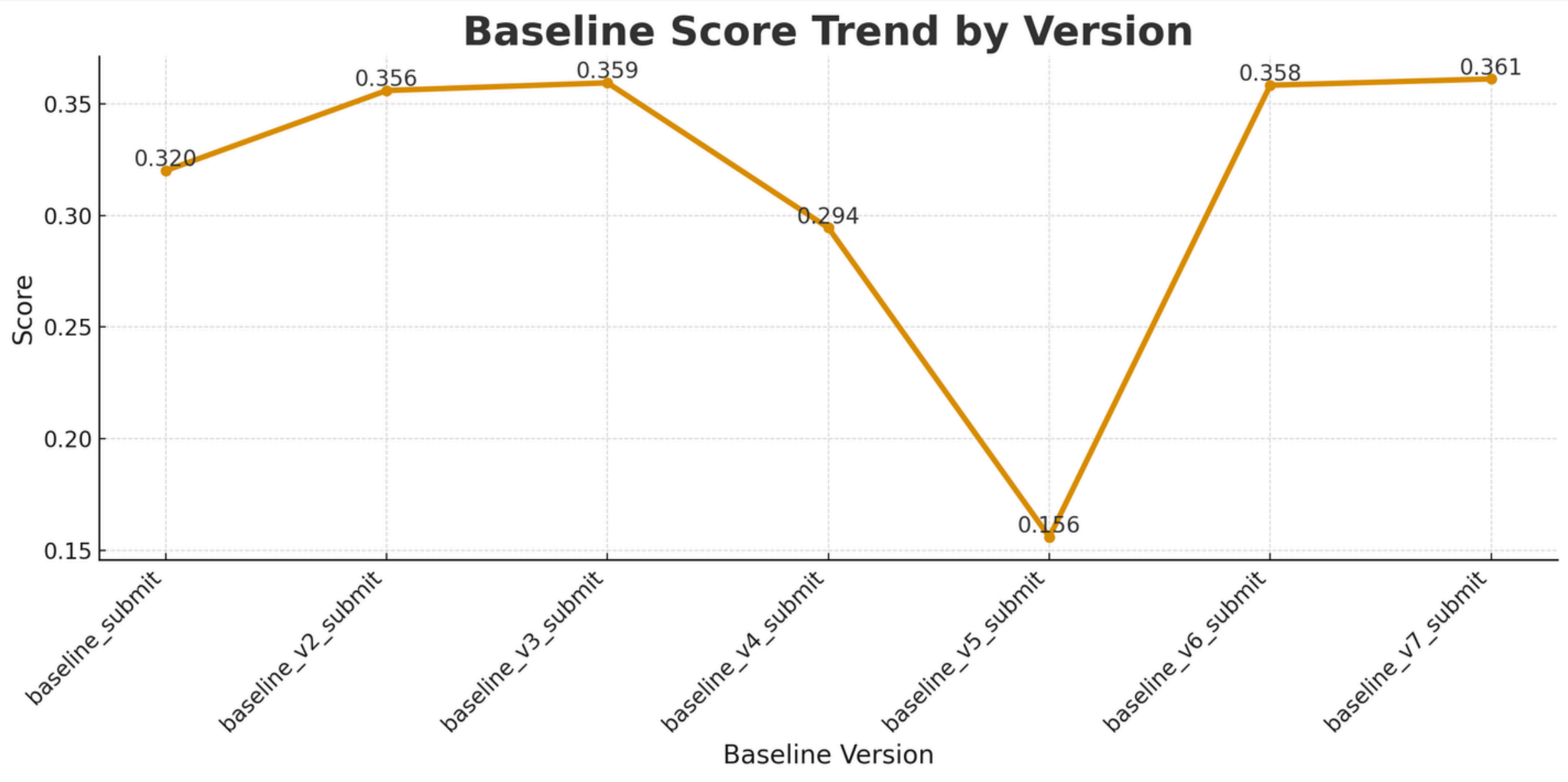
	A	B	C	D	E	F	G	H	I
1	item_id	year	month	seq	type	hs4	weight	quantity	value
2	DEWLVSF	2022	1	1	1	3038	14858	0	32688
3	ELQGMQV	2022	1	1	1	2002	62195	0	110617
4	AHMDUIL	2022	1	1	1	2102	18426	0	72766
5	XIPPENFQ	2022	1	1	1	2501	20426	0	11172
6	FTSVTTSR	2022	1	1	1	2529	248000	0	143004
7	XMKRPGLE	2022	1	1	1	2529	558000	0	205676
8	RCBZUSIM	2022	1	1	1	2805	0	0	459
9	SUOYXCHF	2022	1	1	1	2805	0	0	482
10	ZKENOUD	2022	1	1	1	2805	2	0	2783
11	WQMVCOD	2022	1	1	1	2805	187	0	217777
12	BSRMSVTC	2022	1	1	1	2805	111	0	33952
13	DDEXPPXL	2022	1	1	1	2807	387	0	13158
14	LLHREMKS	2022	1	1	1	2807	319	0	8705
15	DNMPSKT	2022	1	1	1	2811	1748972	0	3961879
16	WPQXWHY	2022	1	1	1	2811	2367757	0	4518277
17	VWMBASN	2022	1	1	1	2811	94038	0	48737
18	RJGPVEXX	2022	1	1	1	2811	18029	0	256916
19	ATLDMDB	2022	1	1	1	2814	68878617	0	60548596
20	SAHWCZN	2022	1	1	1	2825	20305	0	505311
21	LUENUFGA	2022	1	1	1	2833	54965	0	271528
22	HXYSSRXE	2022	1	1	1	2836	5001891	0	2136993
23	QRKRBYJL	2022	1	1	1	2841	30205	0	1766040
24	FRHNWLN	2022	1	1	1	2846	14696	0	529185

데이터 개요

- 총 10,836행 × 9열
- 시계열 무역 데이터 구조
- item_id 기준으로 월별 거래 흐름 확인 가능

무역 데이터 특성

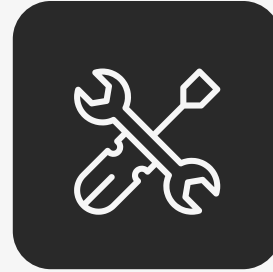
- HS4 기반 상품 분류 제공
- 중량·거래량·거래금액 포함
- 단가 분석·경쟁력 분석 등 무역 특화 분석 가능



전체 모델 흐름 Summary



월별 value 시계열 생성



모든 품목쌍 간 공행성 탐지
(corr + lag 분석)



공행성이 강한 쌍만
feature engineering



회귀 모델로 다음 달 무역량 예측



후처리 후 submission 생성

Baseline Modeling 전체 구조 개요

- 대회 과제 핵심 목표
 - 품목 간 공행성(comovement) 판별
 - 선행 품목 기반 후행 품목의 다음 달 value 예측
- 본 베이스라인은 “데이터 전처리 → 공행성 탐색 → 시계열 모델링 → 제출 파일 생성”까지 전체 프로세스를 구현한 최소 단위 full pipeline

Baseline Pipeline Overview

- Raw train.csv → 월별 item_id 시계열 생성
- 공행성(comovement) 분석 → 선행(lag) 관계 탐색
- Feature engineering → 선형 회귀 모델 학습
- 2025-08 value 예측 → 제출 파일 생성

1. 월별 시계열(pivot) 생성

- 시계열 구성 – item_id 월별 value 집계
- seq/type 등 세부 구분 제거
- item_id 기준 월별 value 총합
- year+month → datetime 변환
- item_id × 월 pivot 테이블 생성 (wide-format)

```
train = pd.read_csv('./train.csv')

monthly = (
    train
    .groupby(["item_id", "year", "month"], as_index=False)["value"]
    .sum()
)

monthly["ym"] = pd.to_datetime(
    monthly["year"].astype(str) + "-" + monthly["month"].astype(str).zfill(2)
)

pivot = (
    monthly
    .pivot(index="item_id", columns="ym", values="value")
    .fillna(0.0)
)

print("pivot shape:", pivot.shape)
print(pivot.head())
```

2. 공행성(comovement) 탐색 알고리즘

- 공행성 : 단순한 상관관계가 아니라 “A가 선행하여 움직이면 B가 일정 시차(lag)를 두고 따라오는 관계”
-
- lag 1~6 범위에서 $A(t)-B(t+lag)$ 상관계수 계산
- 각 품목쌍의 best_lag 및 max_corr 선택
- corr_threshold=0.4 이상 → 공행성 있는 쌍으로 판정
- 결과 예시
 - leading=A, following=B
 - best_lag=2
 - max_corr=0.47

2.공행성(comovement) 탐색 알고리즘

```

# -----
# 2. 공행성 탐색 함수 (threshold/min_nonzero 조정)
# -----
def safe_corr(x, y):
    if np.std(x) == 0 or np.std(y) == 0:
        return 0.0
    return float(np.corrcoef(x, y)[0, 1])

def find_comovement_pairs(pivot, max_lag=6, min_nonzero=10, corr_threshold=0.3):

    items = pivot.index.to_list()
    months = pivot.columns.to_list()
    n_months = len(months)
    results = []

    for _, leader in tqdm(enumerate(items), total=len(items)):
        x = pivot.loc[leader].values.astype(float)

        if np.count_nonzero(x) < min_nonzero:
            continue

        for follower in items:
            if follower == leader:
                continue

```

```

        y = pivot.loc[follower].values.astype(float)
        if np.count_nonzero(y) < min_nonzero:
            continue

        best_lag = None
        best_corr = 0.0

        for lag in range(1, max_lag + 1):
            if n_months <= lag:
                continue

            corr = safe_corr(x[:-lag], y[lag:])
            if abs(corr) > abs(best_corr):
                best_corr = corr
                best_lag = lag

        if best_lag is not None and abs(best_corr) >= corr_threshold:
            results.append({
                "leading_item_id": leader,
                "following_item_id": follower,
                "best_lag": best_lag,
                "max_corr": best_corr,
            })

    return pd.DataFrame(results)

pairs = find_comovement_pairs(pivot)
print("탐색된 공행성쌍 수:", len(pairs))
print(pairs.head())

```

3.시계열 학습 데이터 구성 (Feature Engineering)

- 예측을 위한 시계열 입력 구성
- 입력 feature:
 - b_t: B 현재값
 - b_t_1: B 직전값
 - a_t_lag: A lag 이전값 (선행 효과)
 - max_corr: 공행성 강도
 - best_lag: 최적 lag
- 타겟:
 - b_{t+1}: 다음달 B value

4.모델 학습 (Linear Regression)

- Baseline 모델 - 선형 회귀
- feature 개수가 적고 noise 적음
- baseline 수준에서 안정적인 선택
- 과적합 위험 낮음
- (추후 LightGBM/XGBoost/LSTM으로 확장 가능)

```
def build_training_data(pivot, pairs):
    rows = []
    months = pivot.columns.to_list()
    n_months = len(months)

    for row in pairs.iteruples(index=False):
        leader = row.leading_item_id
        follower = row.following_item_id
        lag = int(row.best_lag)
        corr = float(row.max_corr)

        a_series = pivot.loc[leader].values.astype(float)
        b_series = pivot.loc[follower].values.astype(float)

        for t in range(max(lag, 1), n_months - 1):

            rows.append({
                "b_t": b_series[t],
                "b_t_1": b_series[t - 1],
                "a_t_lag": a_series[t - lag],
                "max_corr": corr,
                "best_lag": float(lag),
                "target": b_series[t + 1], # raw target (log 변환은 아래에서)
            })

    return pd.DataFrame(rows)
```

```
feature_cols = ['b_t', 'b_t_1', 'a_t_lag', 'max_corr', 'best_lag']

train_X = df_train_model[feature_cols].values
train_y = df_train_model["target"].values

# 🔥 log1p 변환 (heavy tail 완화)
train_y_log = np.log1p(train_y)

# LightGBM 모델
reg = LGBMRegressor(
    n_estimators=600,
    learning_rate=0.04,
    max_depth=7,
    subsample=0.9,
    colsample_bytree=0.9
)

reg.fit(train_X, train_y_log)
```

5. 2025년 8월 예측 로직



- 다음달 value 예측 방식
- pivot 마지막 월 = 2025-07
- b_t = B(2025-07)
- b_t_1 = B(2025-06)
- a_t_lag = A(2025-07 - lag)
- 모델 입력 → B(2025-08) 예측

6. 예측 후처리

- 음수 값 → 0으로 보정
- 무역 특성 고려 → 정수 반올림
- 샘플 제출 형식에 맞게 변환

7. 제출 파일 생성

- 공행성 판정된 모든 선행-후행 품목쌍(A→B)에 대해
 - 예측된 다음달 value 저장
- baseline_submit.csv 형태로 출력
- 대회 제공 sample_submission 형식과 완전 일치

```
def predict(pivot, pairs, reg):

    months = pivot.columns.to_list()
    n_months = len(months)

    t_last = n_months - 1
    t_prev = n_months - 2

    preds = []

    for row in tqdm(pairs.itertuples(index=False), total=len(pairs)):

        leader = row.leading_item_id
        follower = row.following_item_id
        lag = int(row.best_lag)
        corr = float(row.max_corr)

        a_series = pivot.loc[leader].values.astype(float)
        b_series = pivot.loc[follower].values.astype(float)

        if t_last - lag < 0:
            continue

        X_test = np.array([[
            b_series[t_last],
            b_series[t_prev],
            a_series[t_last - lag],
            corr,
            float(lag)
        ]])

        # log예측 → expm1 복원
        y_pred_log = reg.predict(X_test)[0]
        y_pred = np.expm1(y_pred_log)
```

version 2 (LightGBM + log1p + threshold 최적화)

1. Linear Regression → LightGBM

- 복잡한 비선형 관계 반영
- 품목별 급변 / 계절성 학습 가능
- feature 간 상호작용 자동 처리

2. log1p 변환 적용

- value 분포: heavy-tail & 이상치 다수
- 극단값 영향 감소, 모델 안정성 증가

3. 공행성 탐색 기준 최적화

- min_nonzero = 12 → 10
- corr_threshold = 0.4 → 0.35

```
# 4. LightGBM 회귀 + log1p 변환
# -----
feature_cols = ['b_t', 'b_t_1', 'a_t_lag', 'max_corr', 'best_lag']

train_X = df_train_model[feature_cols].values
train_y = df_train_model["target"].values

# 🔥 log1p 변환 (heavy tail 완화)
train_y_log = np.log1p(train_y)

# LightGBM 모델
reg = LGBMRegressor(
    n_estimators=600,
    learning_rate=0.04,
    max_depth=7,
    subsample=0.9,
    colsample_bytree=0.9
)

reg.fit(train_X, train_y_log)

print("LightGBM 학습 완료")
```

1. B 품목 시계열 기반 Feature Engineering

- b_t_2 추가 → 과거 관성 반영
- MA2, MA3 → 이동평균으로 추세 강조
- ratio → 월간 변화율(feature) 추가

2. quantity 정보 추가

- q_t, q_t_1 → 수량 기반 패턴 반영
- a_q_lag → 선행 품목 수량 효과 반영

3. Feature 확장

- 5개 → 12개로 확장
- LightGBM + log1p 구조 유지
- 추세 + 변동성 + 수요 신호 동시 학습

```
# B 시계열 기반 feature Engineering
```

```
b_t = b_val[t]
```

```
b_t_1 = b_val[t-1]
```

```
b_t_2 = b_val[t-2]
```

```
ma2 = (b_val[t] + b_val[t-1]) / 2
```

```
ma3 = (b_val[t] + b_val[t-1] + b_val[t-2]) / 3
```

```
ratio = b_val[t] / (b_val[t-1] + 1)
```

```
# quantity feature
```

```
q_t = b_qty[t]
```

```
q_t_1 = b_qty[t-1]
```

```
a_q_lag = a_qty[t - lag]
```

```
feature_cols = [
```

```
    "b_t", "b_t_1", "b_t_2",
```

```
    "ma2", "ma3", "ratio",
```

```
    "q_t", "q_t_1", "a_t_lag", "a_q_lag",
```

```
    "max_corr", "best_lag"
```

```
]
```


version 4 (quantity 기반 공행성 탐색 + LightGBM + log1p + FE)

1. 공행성 탐색 로직 업그레이드

- value 상관 + quantity 상관 결합
- $\text{combined_corr} = 0.7 \cdot \text{value} + 0.3 \cdot \text{quantity}$
- 수량 흐름까지 반영한 복합 공행성 점수 도입

2. 공행성 필터 기준 조정

- $\text{min_nonzero} = 8$
- $\text{corr_threshold} = 0.30$
- 비활성/노이즈 품목 배제 강화

3. Feature Engineering / 모델 구조 유지

- V3 시계열 FE 그대로 유지
- LightGBM + log1p 구조 유지
- “관계 탐색 로직”에 집중한 버전

※ V4 성능 하락 원인 분석

- quantity 시계열의 불규칙성으로 노이즈 증가
 - $\text{value} \neq \text{quantity}$ 시차 패턴 혼재 → 가중 평균 왜곡
 - 느슨한 threshold로 저품질 pair 증가
- value 중심 공행성이 더 안정적

```
corr_v = safe_corr(x_v[:-lag], y_v[lag:])
corr_q = safe_corr(x_q[:-lag], y_q[lag:])

combined = alpha * corr_v + (1 - alpha) * corr_q
```

```
def find_comovement_pairs_v4(
    pivot_v, pivot_q,
    max_lag=6,
    min_nonzero=8,
    corr_threshold=0.30,
    alpha=0.7    # value correlation weight
```

1. 공행성 탐색 보수적 강화

- corr_threshold 0.30 → 0.45
- min_nonzero 10 → 12
- $0.85 \cdot \text{value} + 0.15 \cdot \text{quantity}$

2. 구조

- V3 Feature 유지
- LightGBM + log1p 동일
- “pair 정제”에 집중한 버전

※ 성능하락 원인

기준 과도 → pair 급감 → 학습 신호 부족 → 성능 저하

```
def find_comovement_pairs(  
    pivot_value, pivot_qty,  
    max_lag=6,  
    min_nonzero=12,  
    corr_threshold=0.45,  
):
```

```
corr_val = safe_corr(x_val[:-lag], y_val[lag:])  
corr_qty = safe_corr(x_qty[:-lag], y_qty[lag:])  
  
# value 85% + quantity 15%  
corr_mix = 0.85 * corr_val + 0.15 * corr_qty
```

기존 V3의 구조 유지하면서 입력 특성 고도화하여 개선

추가된 핵심 Feature

- diff : 전월 대비 변화량
- pct_change : 상대적 변동률
- rolling mean / std : 단기 추세 + 변동성
- mini-trend : 최근 3개월 기울기

단순 값 기반이 아닌

“변화 패턴 + 흐름” 중심 학습 구조로 확장

→ LightGBM이 시계열의 방향성과 변동성을
동시에 학습하도록 설계

```
def extract_features(a_series, b_series, t, lag):

    # 기본 값
    b_t = b_series[t]
    b_t_1 = b_series[t - 1]
    a_t_lag = a_series[t - lag]

    # diff
    b_diff = b_t - b_t_1
    a_diff = a_t_lag - a_series[t - lag - 1] if (t - lag - 1 >= 0) else 0

    # pct_change
    b_pct = (b_t - b_t_1) / (b_t_1 + 1e-6)
    a_pct = (a_t_lag - a_series[t - lag - 1]) / (a_series[t - lag - 1] + 1e-6) if (t - lag - 1 >= 0) else 0

    # rolling 3개월 mean/std
    def roll(arr, pos):
        if pos < 2:
            return np.mean(arr[:pos+1]), np.std(arr[:pos+1])
        window = arr[pos-2:pos+1]
        return np.mean(window), np.std(window)

    b_ma3, b_vol3 = roll(b_series, t)
    a_ma3, a_vol3 = roll(a_series, t-lag)

    # mini trend (3-month slope)
    def trend(arr, pos):
        if pos < 2:
            return 0.0
        y = arr[pos-2:pos+1]
        x = np.arange(len(y))
        slope = np.polyfit(x, y, 1)[0]
        return slope

    b_trend = trend(b_series, t)
    a_trend = trend(a_series, t-lag)
```


v6의 과도한 Feature 확장으로 발생한 과적합 문제를 해결하기 위해

검증된 핵심 3개 시계열 Feature만 선택적으로 추가

추가된 핵심 Feature

- b_diff: 현재 대비 전월 변화량
- a_diff: 선행 품목의 변화량
- b_pct: 전월 대비 변동률

✓ 불필요한 통계 Feature 제거

✓ variance 감소 + 학습 안정성 확보

✓ NMAE 개선 + Public Score 상승

- v3 Score: 약 0.3599
- v7 Score: 약 0.3611

```
def extract_features(a_series, b_series, t, lag):  
  
    b_t = b_series[t]  
    b_t_1 = b_series[t - 1]  
    a_t_lag = a_series[t - lag]  
  
    # Feature 1) diff  
    b_diff = b_t - b_t_1  
    a_diff = a_t_lag - a_series[t - lag - 1] if (t - lag - 1 >= 0) else 0.0  
  
    # Feature 2) pct_change  
    b_pct = (b_t - b_t_1) / (b_t_1 + 1e-6)  
  
    return {  
        "b_t": b_t,  
        "b_t_1": b_t_1,  
        "a_t_lag": a_t_lag,  
        "b_diff": b_diff,  
        "a_diff": a_diff,  
        "b_pct": b_pct,  
    }
```